

Face Recognition

1. Experiment Objective

Use the OpenCV Haar Cascade classifier to detect faces in the camera image, with support for detection scaling, histogram equalization, and dynamic parameter tuning.

2. Experiment Principle

1. Key Terms

Term	Description
Haar Cascade	A machine learning object detection method based on Haar features; a sliding window + feature matching approach that quickly locates face regions
detectMultiScale	The core detection function of CascadeClassifier; scans the image with a multi-scale sliding window and returns the positions and sizes of all matched targets
Histogram Equalization	Stretches the grayscale image histogram to the full range, enhancing contrast and improving detection in dark/overexposed environments
Detection Scaling	First shrinks the image (default 0.75×) before detection, greatly reducing computation; the result coordinates are mapped back to the original image proportionally
minNeighbors	Controls detection strictness: larger values mean fewer false positives but possible missed detections; smaller values mean a higher detection rate but more false positives

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start face recognition
```

Data Flow: Image subscription → image conversion → orientation correction → detection scaling → grayscale + `equalizeHist` → `detectMultiScale(scaleFactor=1.1, minNeighbors=5, minSize=(60,60))` → coordinate inverse mapping → draw rectangles + count → real-time display.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☑ Supported
No-PTZ Version	☑ Supported
Required Peripherals	ESP32 camera

2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Face Recognition (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_face.launch.py
```

4. Operation Instructions

Workflow: Image scaling (×0.75) → grayscale + `equalizeHist` histogram equalization → `detectMultiScale(scaleFactor=1.1, minNeighbors=5, minSize=(60,60))` → coordinate denormalization (÷0.75) → `cv2.rectangle` draws the face box + count label.

Dynamic parameter tuning:

```
ros2 param list /detect_face_node # list this node's adjustable parameters
ros2 param set /detect_face_node face_detection_scale 0.5 # at runtime, change face_detection_scale to 0.5 (image scaling ratio before face detection)
```

Adjustable parameters: face detection scaling ratio, minimum number of neighbors, minimum face size, histogram equalization toggle, image flip/rotation, text overlay style, etc.

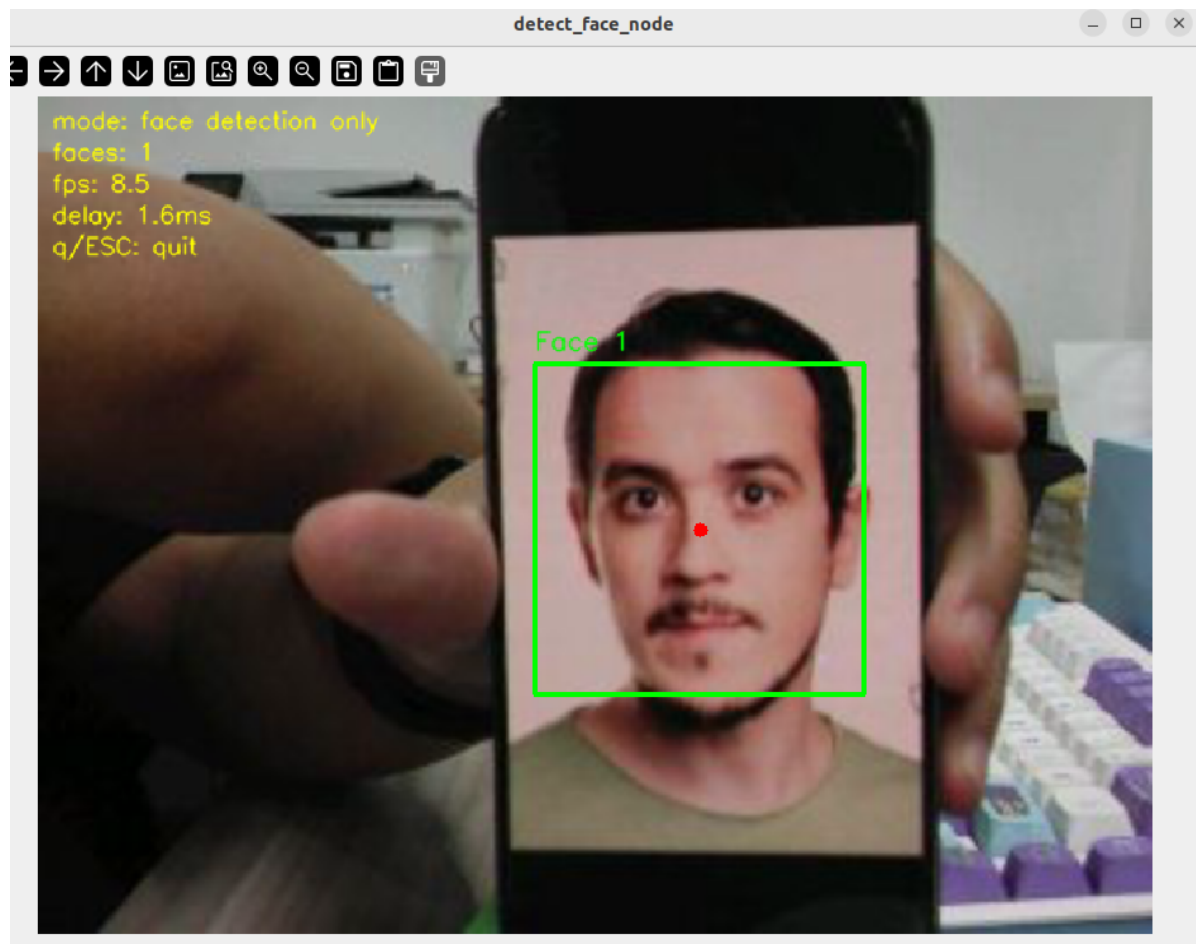
Operation	Function
q / ESC	Exit

5. How to Stop

Press `Ctrl+C` to stop the terminals one by one.

4. Experiment Observations

- **Normal detection:** Faces in the frame are marked with blue rectangles, with a `Face N` index above the box and a face count overlaid in the upper-left corner
- **Multiple faces:** Multiple faces are marked simultaneously, each independently numbered
- **Dark environment:** Histogram equalization automatically enhances contrast, improving detection in low light



5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_face.launch.py`
- Node:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_face.py`

1. Core Node Analysis

`DetectFaceNode` uses the Haar Cascade classifier to detect faces, with support for detection scaling and histogram equalization.

```

# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/de
tect_face.py
class DetectFaceNode(Node):
    def __init__(self):
        super().__init__('detect_face_node')
        self.declare_parameter('image_topic', '/camera/image_raw')
        self.declare_parameter('face_detection_scale', 0.75)
        self.declare_parameter('face_min_neighbors', 5)
        self.declare_parameter('face_min_width', 60)
        self.declare_parameter('face_min_height', 60)
        self.declare_parameter('enable_gray_equalize', True)

        # Prefer the cascade file bundled in the package; fall back to the
OpenCV system built-in
        pkg_share = get_package_share_directory('microros_v2_opencv_visual_pkg')
        cascade_path = os.path.join(pkg_share, 'config',
'haarcascade_frontalface_default.xml')
        if not os.path.exists(cascade_path):
            cascade_path = cv.data.haarcascades +
'haarcascade_frontalface_default.xml'
        self.face_cascade = cv.CascadeClassifier(cascade_path)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        # Detection scaling: shrink the image to reduce computation
        scale = self.get_parameter('face_detection_scale').value
        small = cv.resize(frame, (0, 0), fx=scale, fy=scale)
        gray = cv.cvtColor(small, cv.COLOR_BGR2GRAY)

        if self.get_parameter('enable_gray_equalize').value:
            gray = cv.equalizeHist(gray)

        faces = self.face_cascade.detectMultiScale(
            gray, scaleFactor=1.1, minNeighbors=self.min_neighbors,
            minSize=(self.min_width, self.min_height))

        # Map coordinates back to the original image size
        for (x, y, w, h) in faces:
            x, y, w, h = int(x/scale), int(y/scale), int(w/scale), int(h/scale)
            cv.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)
        return frame

```

Key logic notes:

- **Detection scaling:** with `face_detection_scale=0.75`, the original image is shrunk to 75% before being fed to the Haar Cascade, cutting the computation by about 44% (the area halves); coordinates are mapped back to the original image via `÷0.75`
- **Cascade file fallback:** the custom cascade file bundled in the package `config/` is loaded first; if loading fails, it automatically falls back to the OpenCV system built-in `haarcascades`, ensuring the node can start in any environment
- **Histogram equalization:** `cv2.equalizeHist` stretches the grayscale histogram to the full 0~255 range, significantly improving dark/backlit scenes
- **detectMultiScale parameters:** `scaleFactor=1.1` enlarges the scanning window by 10% at each pass; `minNeighbors=5` requires at least 5 adjacent window confirmations for a valid detection

- The node registers a ROS2 dynamic parameter callback, so runtime parameters can be modified with `ros2 param set /detect_face_node <parameter> <value>`

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_face.launch.py`
- Node:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_face.py`

Parameter	Type	Default Value	Description
<code>face_detection_scale</code>	float	0.75	Scaling ratio before detection (smaller is faster but less accurate)
<code>face_min_neighbors</code>	int	5	Minimum number of neighbors for detectMultiScale
<code>face_min_width</code> / <code>face_min_height</code>	int	60 / 60	Minimum face size (px)
<code>enable_gray_equalize</code>	bool	true	Whether to enable histogram equalization

3. Node Description

Node	Function
<code>detect_face_node</code>	Haar Cascade face detection + histogram equalization + detection scaling optimization + rqt dynamic tuning

6. Frequently Asked Questions

Problem	Cause	Solution
No face detected	Insufficient lighting or the face is too far away	Enable <code>enable_gray_equalize</code> and move the person closer to the camera
False positives (non-face regions boxed)	minNeighbors too low	Increase <code>face_min_neighbors</code> (e.g. 6~8)
Slow detection	Detection scaling ratio too high	Lower <code>face_detection_scale</code> (e.g. 0.5)

